Git & GitHub 学习教程

从入门到精通 · 含 8 周实战学习规划

版本控制 · 远程协作 · 团队工作流 · CI/CD

编写：小强强（WorkBuddy AI 助理）

适用对象：零基础到进阶开发者 / 学生 / 工程师

生成日期：2026-10-27

目录

- 第 1 章 为什么一定要学 Git 与 GitHub
- 第 2 章 Git 核心概念（三区、提交、分支、HEAD）
- 第 3 章 安装与初始化配置（Windows 实战）
- 第 4 章 基础命令实战：第一个仓库
- 第 5 章 分支与合并：冲突解决
- 第 6 章 远程协作基础：GitHub 与 push/pull
- 第 7 章 标准协作流程：Fork / Pull Request / Issue
- 第 8 章 进阶命令：reset / rebase / stash / tag
- 第 9 章 GitHub 生态：Actions / Pages / Releases
- 第 10 章 团队工作流：Git Flow 与 GitHub Flow
- 第 11 章 最佳实践与常见坑
- 第 12 章 8 周学习规划（路线图）
- 第 13 章 命令速查表
- 第 14 章 推荐学习资源

第 1 章 为什么一定要学 Git 与 GitHub

Git 是当今最主流的分布式版本控制系统（VCS），GitHub 则是基于 Git 的全球最大代码托管与协作平台。无论你是写代码、写文档、做数据分析，还是管理项目，掌握它们几乎是“必修课”。

它能帮你解决什么

- 不再丢失历史：每一次修改都被记录，随时可回退到任意版本。
- 多人并行不打架：分支机制让多人同时开发互不干扰，再合并。
- 备份即同步：代码推到远程，电脑坏了也不怕丢。
- 协作有迹可循：谁改了什么、为什么改，一目了然（Code Review）。
- 简历含金量：GitHub 上的开源贡献是技术能力的硬证明。

Git 与 GitHub 的区别

维度	Git	GitHub
是什么	本地命令行工具 / 版本控制引擎	网站 / 云端托管 + 协作平台
运行位置	你自己的电脑	远程服务器（浏览器访问）
核心能力	提交、分支、合并、回退	远程仓库、PR、Issue、Actions、Pages
关系	发动机	车库（存放 + 展示 + 协作）

简单说：Git ■■■■GitHub ■■■■■■■■■■■■■■本地用 Git 管理，推到 GitHub 上备份和协作。

第 2 章 Git 核心概念

理解下面四个概念，Git 就学会了一半。

1. 三个工作区域

区域	英文	说明
工作区	Working Directory	你实际编辑文件的地方（能看到的项目文件夹）
暂存区	Staging Area / Index	“ 待提交清单 ”，决定本次提交包含哪些改动
版本库	Repository (.git)	Git 真正存储所有提交历史的地方

典型流向：■■■ → `git add` → ■■■ → `git commit` → ■■■。

2. 提交（Commit）

一次提交 = 项目在某个时间点的完整快照，带一个唯一 SHA-1
哈希值、作者、时间和说明（message）。提交之间串联成一条时间线。

3. 分支（Branch）

分支是指向某次提交的 “ 可移动指针 ”。默认分支通常叫 `main`（旧版叫
master）。开新分支等于从当前点拉一条新线，互不影响，最后再合并（merge）。

4. HEAD

HEAD
是一个 “ 你现在站在哪 ” 的指针，通常指向当前分支的最新提交。切分支、回退版本，本质都是在移动
HEAD。

第 3 章 安装与初始化配置

Windows 安装步骤

- 下载官方安装包 Git for Windows（本环境已为你准备 Git-2.56.0-64-bit.exe）。
- 双击运行，基本一路“ Next ”即可；关键两项建议保持默认：
-
  - 调整 PATH 环境：选“ Git from the command line and also from 3rd-party software ”。
-
  - 行尾符：选“ Checkout Windows-style, commit Unix-style line endings ”（推荐）。
- 安装完成后，打开“ Git Bash ”或“ 命令提示符 ”，输入 `git --version` 验证。

首次使用必须做的全局配置

```
# ■■ Git ■■■■■■■■■■■■
git config --global user.name  "■■■■"
git config --global user.email "■■■■@example.com"

# ■■■■■■■■■■■■■
git config --global color.ui auto

# ■■■■■■■
git config --list
```

配置 SSH 密钥（推荐，一劳永逸免输密码）

```
# 1. ■■■■■■■■■■■■■
ssh-keygen -t ed25519 -C "■■■■@example.com"

# 2. ■■■■■■■
cat ~/.ssh/id_ed25519.pub

# 3. ■■ GitHub → Settings → SSH and GPG keys
#    ■ New SSH key■■■■■■■■■■■
# 4. ■■■■■
ssh -T git@github.com
```

首次连接会问是否信任，输入 `yes`。看到“ You have successfully authenticated ”即配置成功。

第 4 章 基础命令实战：第一个仓库

场景一：本地新建仓库

```
# ■■■■■■■■■■
cd /d/projects/hello

# ■■■■■■■ .git ■■■
git init

# ■■■■■■■■■■■■
echo "print('hello git')" > main.py

# ■■■■■■■■■■■■ / ■■■■■
git status

# ■■■■■■
git add main.py
# ■■■■■■■■■■■■git add .

# ■■■■■■■
git commit -m "■■■■■■■■■ main.py"
```

场景二：查看历史与差异

```
# ■■■■■■■■■■■■
git log --oneline --graph --all

# ■■■■■■■■■■■■■
git diff

# ■■■■■■■■■■■■■■■
git diff --cached
```

场景三：修改与撤销（日常高频）

```
# ■■■■■■■■■■■■■■■■■■■■■■
git add .
git commit --amend -m "■■■■■"

# ■■■■■■■■■■■■■■■■■■■■■■
git checkout -- ■■■■

# ■■■■■■■■■■■■■■■■
git restore --staged ■■■■
```

第 5 章 分支与合并

分支是 Git 最强大的特性。开发新功能、修 bug，都应该在独立分支上进行。

基本操作

```
# ■■■■■■■* ■■■■■■■
git branch

# ■■■■
git branch feature-login

# ■■■■■■■■■■ switch■
git switch feature-login
# ■■■■“■■■■■”
git switch -c feature-login

# ■■■■■■ main■■■ feature ■■■
git switch main
git merge feature-login

# ■■■■■■■■
git branch -d feature-login
```

冲突解决（必会）

当两条分支改了同一文件的同一处，合并时就会冲突。Git 会标记冲突区域：

```
<<<<<<< HEAD          # ■■■■■■■
print("■■A")
=======               # ■■■
print("■■B")         # ■■■■■■■■■■
>>>>>>> feature-login
```

解决步骤：手动编辑文件，保留你想要的内容并删掉 `<<<<<<<` 等标记 → `git add ■■■` → `git commit` 完成合并。

第 6 章 远程协作基础

在 GitHub 上创建仓库

- 登录 github.com → 右上角“ + ” → New repository。
- 填仓库名（如 hello）、选 Public/Private，不要勾“ Initialize with README ”（本地已有内容时）。
- 创建后，GitHub 会给出远程地址（HTTPS 或 SSH）。

关联远程并推送

```
# ■■■■■■■■■■■■origin ■■■■■■■■
git remote add origin git@github.com:■■■■■■/hello.git

# ■■■■■■-u ■■■■■■■■■■ git push
git push -u origin main

# ■■■■■■■
git push

# ■■■■■■■■■■■■■■■■■■■■■■■■
git fetch
# ■■■■■■
git pull
```

克隆已有仓库

```
# ■■■■■■■■■■■■
git clone git@github.com:■■■■/■■■■.git
# ■■■■■■■■■
git clone ■■ ■■■■■
```

第 7 章 标准协作流程

一、个人项目流程

- 在本地 main 上开发或直接开功能分支。
- add → commit → push 到 GitHub。
- GitHub 上能看到每次提交、可回退、可对比。

二、参与开源 / 团队（Fork + PR 流程）

```
1) Fork■■ GitHub ■■ Fork■■■■■■■■■■■■■■■■■
2) Clone■■■■■■■■■■■■■■■■
      git clone git@github.com:■■■/■■.git
3) ■■■■■■git switch -c fix-bug
4) ■■■■■■■■■■git push -u origin fix-bug
5) ■ GitHub ■■■■ “New Pull Request”
6) ■■■ Review → ■■■Merge■
7) ■■■■■■■■■■
      git remote add upstream ■■■■■■
      git fetch upstream && git merge upstream/main
```

三、Issue 与 .gitignore

- Issue：任务、bug、需求讨论的看板，可指派、打标签、关联 PR。
- .gitignore：告诉 Git 哪些文件不要跟踪（如依赖、日志、密钥）。

```
# .gitignore ■■
node_modules/
__pycache__/
*.log
.env
dist/
```

第 8 章 进阶命令

命令	作用	场景
git reset	移动 HEAD 与分支指针	回退提交（--soft 保留改动，--hard 丢弃）
git revert	生成一个“反向提交”抵消改动	已 push 的提交想撤销，安全不破坏历史
git stash	把当前改动暂存起来	手头活没写完，要切分支修急活
git cherry-pick	把某次提交“摘”到当前分支	只想要别人的某一个提交
git rebase	把提交“搬”到新基点上重放	让提交历史变成一条直线（更整洁）
git tag	给某次提交打标签	标记发布版本 v1.0.0
git amend	修改最后一次提交	提交说明写错 / 漏了文件

rebase 与 merge 的区别

merge 保留真实历史（有合并节点），适合公共分支；rebase 把你的提交“接”到别人后面，历史更线性好看，但不要对已 push 的共享分支做 rebase（会改写历史，给别人添乱）。

stash 示例

```
# ■■■■■■
git stash
# ■■■■■■■■■■…
# ■■■■■■
git stash pop
```

第 9 章 GitHub 生态

功能	一句话说明	用途
GitHub Actions	内置 CI/CD 流水线	自动测试、自动部署，推送即触发
GitHub Pages	免费静态网站托管	把文档 / 博客 / 作品集上线
Releases	发布带安装包的版本	打 tag + 上传二进制，对外发布
Projects	看板式项目管理	类似 Trello，关联 Issue 和 PR
Wiki	项目知识库	写使用文档、设计说明
Gist	代码片段分享	快速贴一小段代码 / 配置
Discussions	社区问答区	用户交流、公告

一个最小 Actions 示例（自动测试）

```
# .github/workflows/test.yml
name: test
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python -m pytest
```

第 10 章 团队工作流

GitHub Flow（推荐新手 / 小团队）

- main 永远可部署。
- 从 main 拉功能分支 → 开发 → 提 PR → Review → 合并 → 部署。
- 简单、快，适合持续交付。

Git Flow（适合有版本发布节奏的团队）

- 长期分支：`main`（生产）、`develop`（开发）。
- 临时分支：`feature/*`（功能）、`release/*`（发布）、`hotfix/*`（紧急修复）。
- 规范但稍重，适合中大型项目。

主干开发（Trunk-Based）

所有人短周期（一天内）合回主干，配合特性开关（feature flag）。高级玩法，依赖强测试和 CI。

第 11 章 最佳实践与常见坑

好习惯

- 提交要“小且完整”，message 写清楚为什么改，不只是改了什么。
- 多用分支，别直接在 main 上乱改。
- push 前先 pull，减少冲突。
- 敏感信息（密码、token）绝不上传，用 .env + .gitignore。
- 定期 fetch，保持和远程同步。

常见坑

- commit 前忘 add：commit 没包含新文件，记得 git add。
- git reset --hard 后文件没了：危险操作，确认无误再执行。
- 对公共分支 rebase：改写历史会让队友崩溃。
- 大文件塞进仓库：用 Git LFS 或外链，否则仓库臃肿。
- 中文文件名/路径乱码：Git Bash 默认 UTF-8，注意终端编码。

第 12 章 8 周学习规划（路线图）

下面是一份循序渐进的路线。每天投入 30 – 60 分钟，8 周可从零基础到能独立协作。

周次	主题	核心目标	本周产出
第 1 周	概念与安装	装好 Git、理解三区/提交/分支概念、完成全局配置与 SSH	本机可运行 git，能推一次到 GitHub
第 2 周	基础命令	熟练掌握 init/add/commit/status/log/diff	本地管理一个练手项目
第 3 周	分支与合并	branch/switch/merge，能解决冲突	在一个仓库里开分支开发与合并
第 4 周	远程协作	remote/push/pull/clone，理解 Fork 与 PR	向一个开源项目提 1 个 PR
第 5 周	协作全流程	Issue、.gitignore、Code Review 规范	用 Issue 管理自己的小项目
第 6 周	进阶命令	reset/revert/stash/cherry-pick/rebase/tag	能安全地回退与整理历史
第 7 周	GitHub 生态	Actions 自动测试、Pages 上线、Releases	项目带 CI 并能一键发布
第 8 周	团队工作流	Git Flow / GitHub Flow，真实协作演练	完整走一遍团队开发流程

每日节奏建议

- 看 20 分钟教程/文档 → 动手 30 分钟实操 → 记一句“今天学会的命令”。
- 第 4 周起，把日常小项目都放进 GitHub，强制自己用分支和 PR。
- 遇到报错先读错误信息，再用英文关键词搜索，比死记命令更管用。

第 13 章 命令速查表

类别	命令	说明
配置	git config --global user.name "x"	设置提交用户名
配置	git config --global user.email "x"	设置提交邮箱
仓库	git init	初始化本地仓库
仓库	git clone	克隆远程仓库
仓库	git status	查看当前状态
提交	git add / .	加入暂存区
提交	git commit -m "msg"	提交到版本库
提交	git log --oneline --graph	查看提交历史
分支	git branch / -d	列出 / 删除分支
分支	git switch / -c	切换 / 新建并切换分支
分支	git merge	合并分支
远程	git remote add origin	关联远程仓库
远程	git push [-u origin main]	推送到远程
远程	git pull	拉取并合并远程更新
远程	git fetch	仅拉取不合并
撤销	git restore --staged	撤出暂存区
撤销	git checkout --	丢弃工作区改动
撤销	git revert	安全撤销已推送提交
暂存	git stash / pop	暂存 / 恢复改动
标签	git tag v1.0 / push --tags	打标签并推送

第 14 章 推荐学习资源

- 官方文档：git-scm.com/doc（权威、全中文）。
- Pro Git（免费电子书）：git-scm.com/book，系统讲透原理。
- GitHub Docs：docs.github.com，平台功能最全。
- 交互式练习：learngitbranching.js.org，用游戏学分支。
- GitHub 官方技能：github.com/skills，手把手仓库教程。
- 中文社区：掘金 / 知乎搜索“ Git 实战 ”，有大量图解。

给小强的一句话

别背命令，先理解“ 三区 + 分支 + 远程 ”三件事；剩下的就是遇到问题查手册。把本周的小项目推上 GitHub，你就已经入门了。加油

—— 本教程由 WorkBuddy「小强强」为你生成